Processes

Chapter 4

Sam Ogden (slides by Glenn Brunns)

OSTEP 04

Updated 2026-05-13 11:07

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Lecture Objectives

After this lecture, you should be able to:

- Define a process, and describe the difference between a program and a process.
- Name 4 process states.
- List the steps the OS uses to initialize a process.
- List a few items contained in a Linux `task_struct`.
- Use 2 bash commands to list processes.

Key Topics

- Processes
- Virtualization
- Context Switch
- Scheduling Policy
- Process State

Key Terms

- Machine State
- Program Counter
- Stack Pointer
- Process States
- Proc Struct
- Process Identifier (PID)

Programs & Processes

What is a process?

- Processes are running programs.
- They have memory and a thread of execution.
- A process is a “running program”, or an “executing program”.
- An OS manages the processes on a computer.
- For example:
  - create a process
  - destroy a process

Program versus Process

Program

- Just the compiled code
  - does not change
  - just a byte stream on disk
  - every copy is identical

Process

- Wraps around a program
  - has a current “state of execution”:
    - what’s the next statement to run?
    - what are the values in memory?
    - what files are open?
  - multiple copies of a program can be running at once
    - each copy is unique

Virtualization

Why processes?

- Processes allow us to share resources by considering programs as abstract objects.
- Allows us to reason about processes more cleanly.
- Allows us to pretend we have one computer per process.

Process Management

OS data structures for process mgmt.

- For each process there is an OS data structure containing things like:
  - the process id
  - the process state
  - the process register values
  - the size of process memory
- In Linux, the data structure is called `task_struct`.
- When a process is created, a new `task_struct` is allocated.

OS data structures for process mgmt.

Context Switching saves all of this to memory

```
// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// the information xv6 tracks about each process
// including its register context and state
struct proc {
  char *mem;                  // Start of process memory
  uint sz;                    // Size of process memory
  char *kstack;               // Bottom of kernel stack
                              // for this process
  enum proc_state state;      // Process state
  int pid;                    // Process ID
  struct proc *parent;        // Parent process
  void *chan;                 // If !zero, sleeping on chan
  int killed;                 // If !zero, has been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;          // Current directory
  struct context context;     // Switch here to run process
  struct trapframe *tf;       // Trap frame for the
                              // current interrupt
};
```

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
  int eip;
  int esp;
  int ebx;
  int ecx;
  int edx;
  int esi;
  int edi;
  int ebp;
};
```

Process states

Four main states we care about

- ready
  - Job has no blockers and is waiting to be chosen by the OS
- running
  - Job is currently on the processor
- blocked
  - Job is waiting on something (e.g. I/O)
- terminated
  - Job is no longer running

Note: “Job” == “Process”

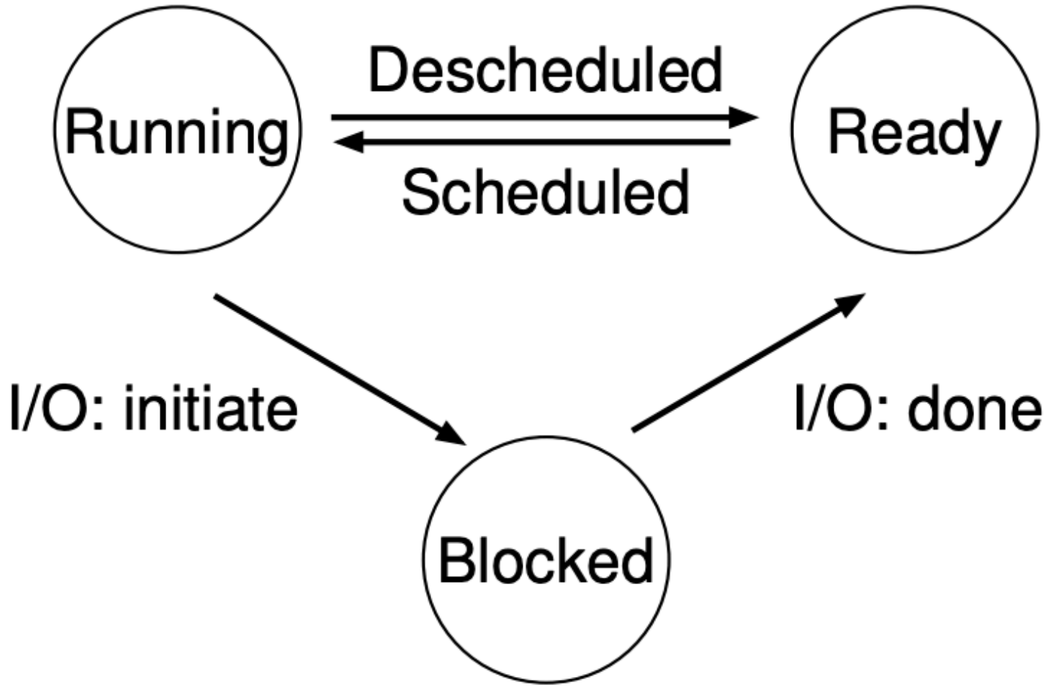
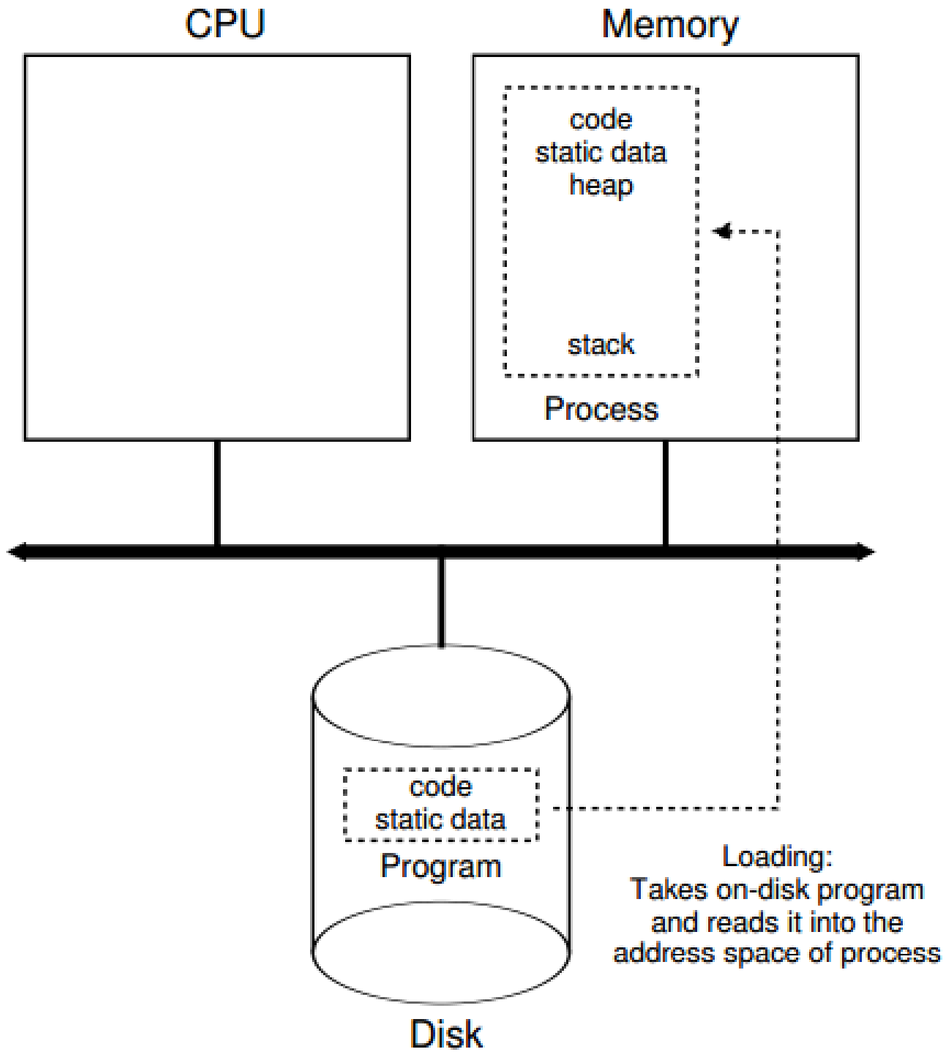


Figure 4.2: **Process: State Transitions**

How does the OS create a process?

1. get the program code off disk, load it into memory
2. load static data into memory
3. allocate memory for stack
4. allocate memory for heap
5. initialize some file descriptors
6. transfer control to the program



Process Scheduling

Two main kinds of Scheduling

Batch Scheduling

- **Typical use case**: There is a set of jobs that needs to be completed
- **General idea**: We pick a job and run it until it's completed or preempted
- **Goal**: We finish all the jobs as quickly as possible

Multi-Programming

- **Typical use case**: One or more users are interacting with processes
- **General idea**: We run each job for a short period of time.
  - It is still one at a time, but if we do it fast enough we can pretend they're running at the same time!
- **Goal**: We are highly responsive

How to pause and resume programs?

- Conceptually, to pause:
  - stop process and record its execution state
- To resume:
  - restore the process execution state, and restart
- We'll get into the details later

Mechanism and Policy

- The OS has a **mechanism** to allow it to stop and start processes. ("*How?*")
- The OS defines a **policy** about how to use the mechanism to achieve goals. ("*Why?*")
  - how long should a process be allowed to run?
  - which processes should get higher priority?
  - how to ensure every process eventually gets a chance to run?

Mechanism vs Policy is a recurring theme in OS design.

Listing processes with bash

```
$ ps
  PID TTY          TIME CMD
 5568 pts/0    00:00:00 bash
 5593 pts/0    00:00:00 ps

$ ps -ef
...
ziel5122  4872  4868  0 15:16 ?        00:00:00 sshd: ziel5122@notty
ziel5122  4873  4872  0 15:16 ?        00:00:00 sshd: ziel5122@internal...
root      5562  4257  0 15:31 ?        00:00:00 sshd: brun1992 [priv]
root      5564     2  0 15:31 ?        00:00:00 [flush-253:0]
brun1992  5567  5562  0 15:31 ?        00:00:00 sshd: brun1992@pts/0
brun1992  5568  5567  0 15:31 pts/0    00:00:00 -bash
apache    5666 10367  0 Sep06 ?        00:00:01 /usr/sbin/httpd
apache    5667 10367  0 Sep06 ?        00:00:01 /usr/sbin/httpd
apache    5668 10367  0 Sep06 ?        00:00:01 /usr/sbin/httpd
...
```

- with no options, `ps` shows processes of current user in current terminal
- with the `-e` (all processes) and `-f` (full output) options:

ps - details

```
$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
...
ziel5122  4872  4868  0 15:16 ?        00:00:00 sshd: ziel5122@notty
ziel5122  4873  4872  0 15:16 ?        00:00:00 sshd: ziel5122@internal...
root      5562  4257  0 15:31 ?        00:00:00 sshd: brun1992 [priv]
root      5564     2  0 15:31 ?        00:00:00 [flush-253:0]
brun1992  5567  5562  0 15:31 ?        00:00:00 sshd: brun1992@pts/0
brun1992  5568  5567  0 15:31 pts/0    00:00:00 -bash
apache    5666 10367  0 Sep06 ?        00:00:01 /usr/sbin/httpd
apache    5667 10367  0 Sep06 ?        00:00:01 /usr/sbin/httpd
apache    5668 10367  0 Sep06 ?        00:00:01 /usr/sbin/httpd
...
```

- user responsible for launching the process
- process ID
- parent process ID
- proc. utilization
- system time at process creation
- terminal from which process launched
- cumulative CPU time for this process
- program running in the process

top - show processes interactively

```
$ top
top - 15:48:16 up 24 days,  9:11,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 196 total,   1 running, 195 sleeping,   0 stopped,   0 zombie
Cpu(s):  0.7%us,  0.7%sy,  0.0%ni, 98.3%id,  0.3%wa,  0.0%hi,  0.0%si,  0.0%st
Mem:   4019552k total,  2737172k used,  1282380k free,   440900k buffers
Swap:  4161532k total,       44k used,  4161488k free,  1845072k cached
PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
2084 root      20   0  121m 9292 4668 S  0.3  0.2   2:23.50 nsrexecd
6520 brun1992  20   0  2704 1180  872 R  0.3  0.0   0:00.19 top
11268 root      20   0 26664 4128 3348 S  0.3  0.1  55:01.68 vmtoolsd
1 root      20   0  2900 1380 1208 S  0.0  0.0   1:06.58 init
2 root      20   0     0    0    0 S  0.0  0.0   0:04.22 kthreadd
3 root      RT   0     0    0    0 S  0.0  0.0   0:00.00 migration/0
4 root      20   0     0    0    0 S  0.0  0.0   0:21.83 ksoftirqd/0
5 root      RT   0     0    0    0 S  0.0  0.0   0:00.00 stopper/0
6 root      RT   0     0    0    0 S  0.0  0.0   0:21.44 watchdog/0
```

- 1, 5, and 15-minute load averages
- %CPU: process’s share of the elapsed CPU time since last screen update
- %MEM: process’s currently-used share of available physical memory

Summary

- A process is a running program.
- A main job of the OS is to manage processes: create them, destroy them, stop them, start them, ...
- Multi-programming is when the OS runs multiple programs “at once” on a single processor.
- Mechanism vs Policy.